

Group 4

Exit the MazeTrix

Software Engineering Project - Sprint 2

Name (s): Rebecca Alvarez, Sean Cho, Jack Lin, Tony Nguyen, Kenny Vo

Semester: Spring 2026

Group Number: 4

Coordinator: Kenny Vo (Sprint 2)

Name of the Guide: Dr. Tushara Sadasivuni

Date: (02/23/2026)

TABLE OF CONTENTS

1.0	<i>PLANNING AND SCHEDULING.....</i>	<i>3</i>
2.0	<i>IMPROVED PROBLEM STATEMENT.....</i>	<i>4</i>
3.0	<i>IMPROVED CONTEXT DIAGRAM.....</i>	<i>6</i>
4.0	<i>IMPROVED ACTIVITY DIAGRAM.....</i>	<i>7</i>
5.0	<i>SYSTEM REQUIREMENTS.....</i>	<i>8</i>
5.1	<i>Use Cases.....</i>	<i>8</i>
5.2	<i>Requirements.....</i>	<i>13</i>
5.1	<i>Use Case Diagrams.....</i>	<i>21</i>
6.0	<i>DATABASE MANAGEMENT.....</i>	<i>22</i>
7.0	<i>IMPLEMENTATION.....</i>	<i>24</i>
7.1	<i>Functional User Interface.....</i>	<i>24</i>
7.2	<i>Visual Mock-Up.....</i>	<i>X</i>

1.0 PLANNING AND SCHEDULING

Assignee Name	Email address	Task	Duration (hours)	Dependancy	Due Date	Evaluation
Rebecca Alvarez	ralvarez3@student.gsu.edu	Login Screen & Main Menu Visual Mockups Improved Problem Statement Improved Activity Diagram Use Cases #4, 5, 7-10	10 hours	None	02/22	100%
Sean Cho	scho55@student.gsu.edu	Requirements 1-10	1-2 hours	None	2/22	100%
Jack Lin	jlin49@student.gsu.edu	Use Case Diagram	3-4 hours	None	2/22	100%
Tony Nguyen	hnguyen301@student.gsu.edu	Use case drafts for # 6, 10 Requirements #1-10	1-2 hours	None	2/22	100%
Kenny Vo (coordinator)	hvo30@student.gsu.edu	Database creation and linking to the log in and sign up feature of the software with Google Firestore	12-13 hours	None	2/22	100%

2.0 IMPROVED PROBLEM STATEMENT

Our product is a maze-based game that challenges players to think strategically while navigating through different maze levels. In addition to finding a way out of the maze, players must manage their health and hunger while dealing with enemies along the way. The game focuses on survival, decision-making, and problem-solving rather than just speed. It includes the following features:

- **Save system**: The user will be prompted to create a login upon opening the game. Their credentials will be connected to a database which also stores the current state of their attempt, score, and leaderboard ranking.
- **AI Generated Maze**: The maze itself is a single, infinitely procedurally generated maze with dynamic placing for enemies, environmental hazards, and pickups. **Generation is based on a seed selected by the OpenAI Narrator.**
- **Minimap**: Will show the current state of the maze within a certain range of the current player location to assist in navigation.
- **OpenAI Narrator**: An AI-powered Narrator will track all of the players activity within the game and comment on their progress.
- **Health**: There are two status bars the player must manage while going through the maze. One is health, which is decreased upon taking damage from enemies, hazards, and low hunger.
- **Hunger**: The second status bar that must be managed by the player. Hunger will decrease over time at a constant rate and can be recovered by finding food items throughout the maze.
- **Enemies**: Will spawn randomly within the maze, and can wander from their initial spawn location. They harm the player at close range, but can be defeated with weapons and drop experience for levels and pickups they die.
- **Audio Detection**: Audio sources attract enemies. This includes in-game audio effects, **as well as audio from the user's microphone to refer to the player location.**
- **Traps & Environmental Hazards**: Stepping on tiles may reveal traps with various effects. These will appear as normal tiles until stepped on, after which they will have a visual difference to prevent recurring activations.
- **Pickups**: Spawn randomly throughout the maze, but do not regenerate to encourage the player to explore new areas. Include Potions to restore health, food to increase hunger, and bombs for tile destruction.
- **Tile Destruction**: The bomb pickup allows the player to destroy walls of the maze, which act as an additional tool the player can use to escape enemies or aid in exploring new portions.
- **Speed Boost**: A special power-up that will temporarily increase the player's speed.
- **Level-Up**: Gaining enough experience from killing enemies will allow the player to upgrade either max health or attack power
- **Dynamic Difficulty**: The health and attack of enemies will increase the higher the player's level and the longer they survive in the maze.

MazeTrix is designed for casual and intermediate gamers who enjoy puzzle and strategy-based games. It is aimed at players who like maze challenges but also want an added layer of survival mechanics, such as managing health and hunger. The game is suitable for students and young adults who are looking for an engaging and mentally challenging gameplay experience.

MazeTrix is designed to be a cure for boredom, an escape from reality. This game solves the repetitive loop of other maze games with its dynamic gameplay and Generative AI features. Every round will feel challenging and failure isn't frustrating, it becomes part of the progression, with each attempt bringing players closer to mastery. MazeTrix fosters a sense of accomplishment and resilience. By encouraging players to keep trying, adapting, and improving, the game helps build habits of persistence, problem-solving, and working toward meaningful goals.

Several existing maze and survival games offer similar gameplay elements. Traditional maze games focus mainly on navigation and puzzle-solving, while survival games emphasize managing health and resources. However, MazeTrix differs by offering both, combining maze navigation, survival mechanics, and strategic decision-making into a single experience.

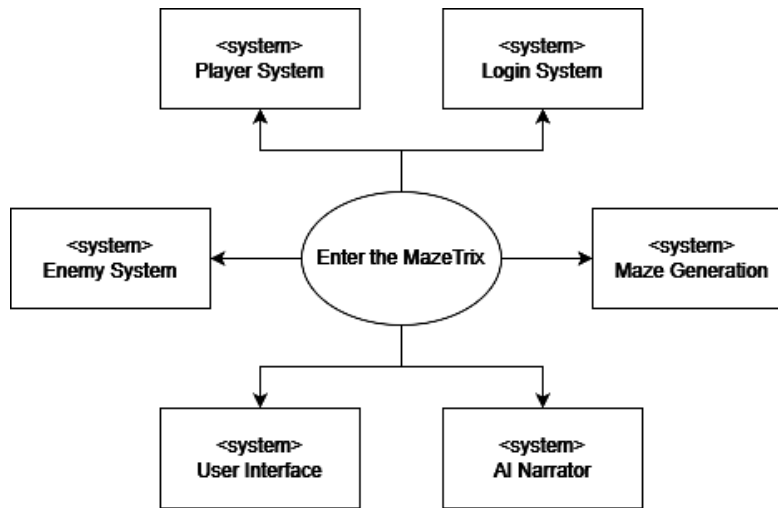
This project is compelling because it combines maze navigation, survival mechanics, and enemy interactions into a single game that is both challenging and engaging. Instead of focusing only on solving a maze, players must also manage limited resources such as health and hunger while facing enemies, which adds depth and challenges. From a development perspective, the project allows the team to apply important software engineering concepts such as game logic, state management, teamwork, and iterative development through sprints. It is worth developing through sprints. It is worth developing because it is both engaging for players and valuable as a learning experience for the team.

The system is realistic and feasible to build using the tools, programming languages, and resources available to the team. Core Features such as player movement, health, and hunger systems, enemy behavior, and user interface elements can be implemented using standard game development techniques and existing libraries. Additionally, developing MazeTrix allows the team to effectively apply software engineering principles such as modular design, version control, and sprint-based development, making good use of both time and technology.

From a technical perspective, this project is interesting because it involves integrating multiple systems into a single, functional game. The development requires implementing maze generation or level design, player movement with collision detection, enemy behavior, and real-time updates to player stats such as health and hunger. The project also involves managing game state, handling user input, and updating the user interface dynamically. Coordinating these components in a modular and scalable way makes the project a strong opportunity to overcome challenges and gain software engineering experience.

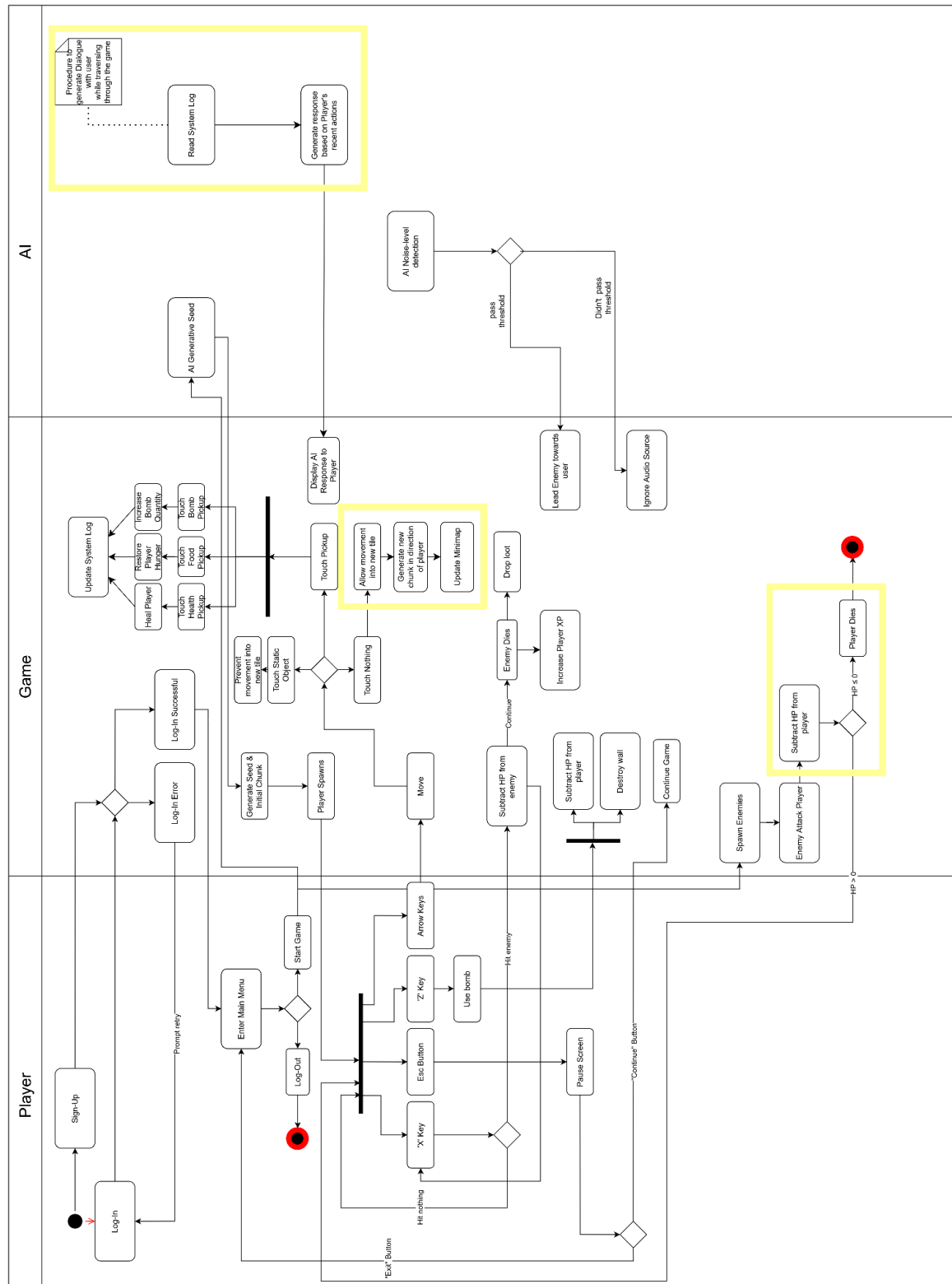
There will indeed be a client login, but there will not be an admin login. This is due to there not being any admin-specific features for this application.

3.0 IMPROVED CONTEXT DIAGRAM



No changes requested.

4.0 IMPROVED ACTIVITY DIAGRAM



5.0 SYSTEM REQUIREMENTS

5.1 Use Cases

Use Case no.: 1

Use Case Name: User Registration

Actors: User, System, Database

Description: This use case describes the process for a new user to create an account. When the application launches, the user is given the option to register. The user enters their chosen credentials. The system validates the input and stores the new account information in the Firestore database. Once registration is successful, the account is created and ready for future logins. The stored credentials will be used to save and retrieve the player's game data

Alternate Path:

- The user chooses to exit instead of registering
- The system closes the application
- No account was created, and no data is saved

Exception Path:

- Username/Email Already Exists:
 - + The database detects an existing account with the same email/username
 - + The system displays: "Email/Username already exists"
 - + The user is redirected to the login screen
- Invalid Input Format:
 - + If required fields are empty or formatted incorrectly
 - + The system displays an appropriate validation message
 - + The user remains on the registration screen
- Database Connection Failure:
 - + If the system cannot connect to Firestore
 - + An error message displays: "Unable to register. Please try again later"
 - + The account is not created

Pre-condition:

- The user does not already have an account
- The system is running properly
- The system is connected to the Firestore database

Post-condition:

- User account is created
- User data is stored in the database

Use Case no.: 2

Use Case Name: User Log-In

Actors: User, System, Database

Description: This use case describes the process by which an existing user logs into the system. The user enters their registered email/username and password. The system validates the credentials against the records stored in the Firestore database. If the credentials match, the user is granted access and redirected to the main menu screen of the application.

Alternate Path: If credentials fail to match, the user will be prompted to try again or be given the option to create an account if they do not have one already made.

Exception Path:

- Database Connection Failure:
 - + If the system cannot connect to Firestore, an error message is displayed
 - + The user remains on the login screen
- Empty Input Fields:
 - + If the user leaves required fields blank, the system prompts user to input missing information
 - + The system does not attempt authentication

Pre-condition:

- The user has already created an account
- The user has access to the login screen
- The system is connected to the Firestore database

Post-condition:

- If successful
 - + The user is authenticated
 - + A session is created for the user
 - + The user is redirected to the main menu
- If unsuccessful
 - + The user remains on the login screen
 - + No session is created

Use Case no.: 3

Use Case Name: Main Menu Navigation

Actors: User, System

Description: This use case describes the interaction between the user and the system at the main menu screen. After launching the game or successfully logging in, the user is presented with the main menu. The menu provides options such as “Start Game” and “Exit Game”. The user selects one of the available options, and the systems responds accordingly

Alternate Path:

- The user selects the “Exit Game” option
- The system confirms the action
- The system closes the application

Exception Path:

- Game Resources Fail to Load:
 - + If required game files or resources fail to initialize
 - + The system displays an error message: “Unable to start game. Please restart the application”
 - + The user remains on the main menu screen
- Unexpected System Error:
 - + If a critical error occurs, the system logs the error and safely returns the user to the main menu or closes the application

Pre-condition:

- The game application is running
- The user has successfully logged in or has reached the main menu screen
- The system is functioning properly

Post-condition:

- If “Start Game” is selected:

- + The game session begins
 - + The system loads the appropriate game environment
 - If “Exit Game” is selected:
 - + The application closes
 - + The sessions ends
-

Use Case no.: 4**Use Case Name:** Player Movement**Actors:** User, Player Object, System**Description:**

- The user presses one of the arrow keys on their keyboard.
- The player object moves in the direction of the key pressed.
- User releases key.
- Player object stops moving.

Alternate Path: There are no Alternate Paths for this use case.**Exception Path:**

- If the path in direction of desired movement is blocked, Player Object will be forced to stop by the obstruction.

Pre-condition:

- User selected ‘Start Game’ on Main Menu
- User has functioning hardware

Post-condition: Player Object will have moved to a different position.

Use Case no.: 5**Use Case Name:** Enemy Hits Player**Actors:** Player Object, Enemy, System**Description:**

- Enemy attack animation plays.
- If the enemy attack lands on the player:
 - + Calculate the damage done
 - + Update Player HP
 - + Determine if Player dies from the attack

Alternate Path: There are no Alternate Paths for this use case.**Exception Path:** If any of the following are true, the attack will not occur:

- Enemy or Player leaves attack range, thus causing the attack to miss.
- Enemy or Player dies before the attack is complete, thus preventing any damage done.

Pre-condition: The player must be within attack range of the enemy.**Post-condition:** Player HP decreases, or if HP is ≤ 0 , then the Player dies and the game ends

Use Case no.: 6**Use Case Name:** Player attack enemy**Actors:** Player, Enemy**Description:** The player initiates a combat action against an enemy within range.

The system calculates the damage, update enemy health points (HP), and determine

if that enemy is defeated

Alternate Path: If the enemy is far away from the player's attack range, or die before that enemy gets hit, no damage would be dealt

Exception Path: If no enemy is around, the system displays "missed attack" message and no damage is applied.

Pre-condition: The player must be in a game session and the enemy must be within the attack range

Post-condition: Enemy's HP is reduced, or the enemy is eliminated if the HP reduces to 0.

Use Case no.: 7

Use Case Name: Player Encounters Trap

Actors: Player Object, System

Description:

- Player moves onto a trap tile
- If Player has activated this trap tile before, it will have a different appearance, otherwise it will blend into the floor
- Trap activates and deals a single instance of damage to the player
- Trap will now be visibly distinct from the other tiles if this is the first activation of the trap

Alternate Path: There are no Alternate Paths for this use case.

Exception Path: If Player dies from other sources before the trap can deal damage, then they take no damage

Pre-condition: Player Movement use case must be completed

Post-condition: Player HP decreases, or if HP is ≤ 0 , then the Player dies and the game ends

Use Case no.: 8

Use Case Name: Player Places Bomb

Actors: Player Object, Enemy, System

Description:

- Player presses the Z key on the keyboard to place a bomb
- The number of bombs in the Player's possession is subtracted by 1
- After 3 seconds, the bomb explodes, leading to the following:
 - + If the Player is nearby, deal damage to the Player
 - + If any enemies are nearby, deal damage to the enemies
 - + If there is a wall tile of the maze nearby, destroy the wall

Alternate Path: There are no Alternate Paths for this use case.

Exception Path: Player presses the 'Z' key without sufficient bombs, then no bomb will be placed.

Pre-condition: Player must have at least 1 bomb in their inventory via Player Item Pickup use case

Post-condition: Dependent on the adjacent targets of the bomb

- Player HP decreases, or if HP is ≤ 0 , then the Player dies and the game ends
- Enemy's HP is reduced, or the enemy is eliminated if the HP reduces to 0.
- Wall tile of the maze is deleted

Use Case no.: 9**Use Case Name:** Enemy detects player audio**Actors:** User, Enemy, System**Description:**

- If User performs any of the following actions:
 - + Player Movement
 - + Player activates a trap
 - + Creates a sound audible by their device microphone
 - + Player places a bomb
- It will be audible to enemies within current chunk, and the enemies will begin to patrol toward the position of the last detected sound
- The sound of a bomb explosion will instead be audible to enemies within the adjacent chunks

Alternate Path:**Exception Path:**

- If a user does not have a functioning audio input device, this is treated as no sound

Pre-condition: At least one of the following use cases:

- #4: Player Movement
- #7: Player Places a Bomb
- #8: Player Activates a Trap

Post-condition: Enemies will begin to patrol toward the position of the last detected sound

Use Case no.: 10**Use Case Name:** Collect items**Actors:** Player, System**Description:**

- Player moves onto a tile containing an item
- The item will disappear and affect the player's statistics as follows:
 - + Health Potion: recovers the Player's missing health
 - + Food: recovers the Player's missing hunger
 - + Bomb: increase the number of held bombs by 1
 - + Speed Boost: double the Player's movement speed for 20 seconds

Alternate Path: There are no Alternate Paths for this use case.**Exception Path:** The item will not be picked up under the following conditions:

- Health Potions if the Player is not missing any health
- Food if the Player is not missing any hunger

Pre-condition: Player Movement use case required for movement onto the tile containing the item**Post-condition:** Player statistics are altered depending on the item picked up

5.2 Requirements

Requirement #: 1**Use Case #: 1****Name:** User Registration**Introduction:**

A clean and simple 2–3 input form that allows new users to create an account before accessing the game.

Rationale:

Users must be able to register in order to save progress, store game data, and securely access their profile.

Input:

- Email / Username
- Password
- Confirm Password

Requirement Description:

The user should be able to create a new account. This is only possible if all requirements are met:

- All required input fields must be completed.
- Email must follow proper format.
- Passwords must meet minimum security requirements (length, etc.).
- Email/Username must not already exist in Firestore.
- The system must successfully connect to the Firestore database.

Output:

- The account is successfully created in the database.
 - The user is redirected to the Login screen or Main Menu.
-

Requirement #: 2**Use Case #: 2****Name:** User Login**Introduction:**

A simple 2-input form that allows an existing user to log into the game and access the Main Menu.

Rationale:

The login process authenticates the user and retrieves their saved game data from Firestore.

Input:

- Email / Username
- Password

Requirement Description:

The user should be able to log in. This is only possible if all requirements are met:

- All required fields must be completed.
- The system validates input format before authentication.
- Credentials must match records stored in Firestore.
- The database connection must be active.

Output:

- The user is authenticated.
 - User session is created.
 - The user is routed to the Main Menu screen.
-

Requirement #: 3**Use Case #: 3**

Name: Main Menu Navigation

Introduction:

A simple main menu screen that allows users to start a game session or exit the application.

Rationale:

The Main Menu acts as the central navigation hub of the application.

Input:

- User selects "Start Game" or "Exit Game"

Requirement Description:

The user should be able to navigate using the main menu options:

- If "Start Game" is selected, the system loads game resources and begins a new session.

- If “Exit Game” is selected, the system safely closes the application.
- The system must verify that required game resources are initialized before starting.

Output:

- Start Game → Game session begins.
 - Exit Game → Application closes.
-

Requirement #: 4**Use Case #: 4**

Name: Player Movement

Introduction:

The player must be able to move the character within the maze using keyboard controls.

Rationale:

Player movement is a core gameplay mechanic required for exploration and interaction.

Input:

- Arrow keys

Requirement Description:

The player should be able to move within the maze environment:

- The system detects keyboard input.
- The player object moves in the corresponding direction.
- Movement stops when the key is released.
- Players cannot move through walls or blocked tiles.

Output:

- Player position updates within the game world.
-

Requirement #: 5**Use Case #: 5**

Name: Enemy Attack Player

Introduction:

The system must allow enemies to attack the player when within attack range.

Rationale:

Enemy attacks introduce challenge and risk, which are essential to core gameplay mechanics.

Input:

- Player enters enemy attack range
- Enemy attack animation triggers

Requirement Description:

The enemy should be able to damage the player if requirements are met:

- The player must be within enemy attack range.
- The enemy must be alive.
- The system must detect collision/hit confirmation.
- The system calculates damage based on enemy stats.
- Player HP is reduced accordingly.
- If Player HP ≤ 0 , the system triggers game over.

Output:

- Player HP decreases.
 - If HP reaches 0, the game session ends.
-

Requirement #: 6**Use Case #: 6**

Name: Player Attack Enemy

Introduction:

The player must be able to initiate combat against enemies within range.

Rationale:

Player combat is required to eliminate enemies and progress through the maze.

Input:

- Player attack command (keyboard input)

Requirement Description:

The player should be able to damage enemies if requirements are met:

- Enemies must be within attack range.
- The player must be alive.
- The system confirms attack hit detection.
- The system calculates damage.
- Enemy HP is reduced accordingly.
- If Enemy HP ≤ 0 , the enemy is removed from the game.

Output:

- Enemy HP decreases.
 - Enemy eliminated if HP reaches 0.
-

Requirement #: 7**Use Case #: 7**

Name: Player Encounters Trap

Introduction:

The system must handle trap activation when a player steps onto a trap tile.

Rationale:

Traps add environmental hazards that increase difficulty and strategy.

Input:

- Player moves onto trap tile

Requirement Description:

The trap should activate if requirements are met:

- The player enters a trap tile.
- The system checks whether the trap has been previously activated.
- Trap deals a single instance of damage.
- Trap becomes visibly distinct after first activation.
- If Player HP ≤ 0 , the game ends.

Output:

- Player HP decreases.
- Trap state changes to activate.

Requirement #: 8**Use Case #: 8****Name:** Bomb Placement**Introduction:**

The player must be able to place bombs to interact with enemies and maze walls.

Rationale:

Bomb placement adds strategic gameplay and enables maze navigation and combat mechanics.

Input:

- Press "Z" key

Requirement Description:

The player should be able to place a bomb if requirements are met:

- Players must have at least one bomb in inventory.
- The system subtracts one bomb from inventory.
- A 3-second fuse timer begins.
- Explosion damages nearby enemies and/or players.
- Explosion destroys destructible walls.

Output:

- The bomb explodes after the timer.
 - Damage applied to valid targets.
 - Wall tiles removed if applicable.
-

Requirement #: 9**Use Case #: 9****Name:** Enemy Audio Detection**Introduction:**

Enemies must detect player-generated sounds and react accordingly.

Rationale:

Audio-based AI detection increases immersion and stealth mechanics.

Input:

- Player movement
- Trap activation
- Bomb placement
- Microphone-detected sound

Requirement Description:

Enemies should react to sound if requirements are met:

- Sound must originate within detectable range or chunk.
- The system identifies the source location of the sound.
- Enemies in range change patrol state to investigate.
- Bomb explosions are detectable across adjacent chunks.
- If no audio device exists, no sound detection occurs.

Output:

- Enemies patrol toward the last detected sound location.
-

Requirement #: 10**Use Case #: 10**

Name: Item Collection

Introduction:

The system must allow players to collect items and update player statistics.

Rationale:

Item collection supports survival, resource management, and progression.

Input:

- Player moves onto item tile

Requirement Description:

The player should collect items if requirements are met:

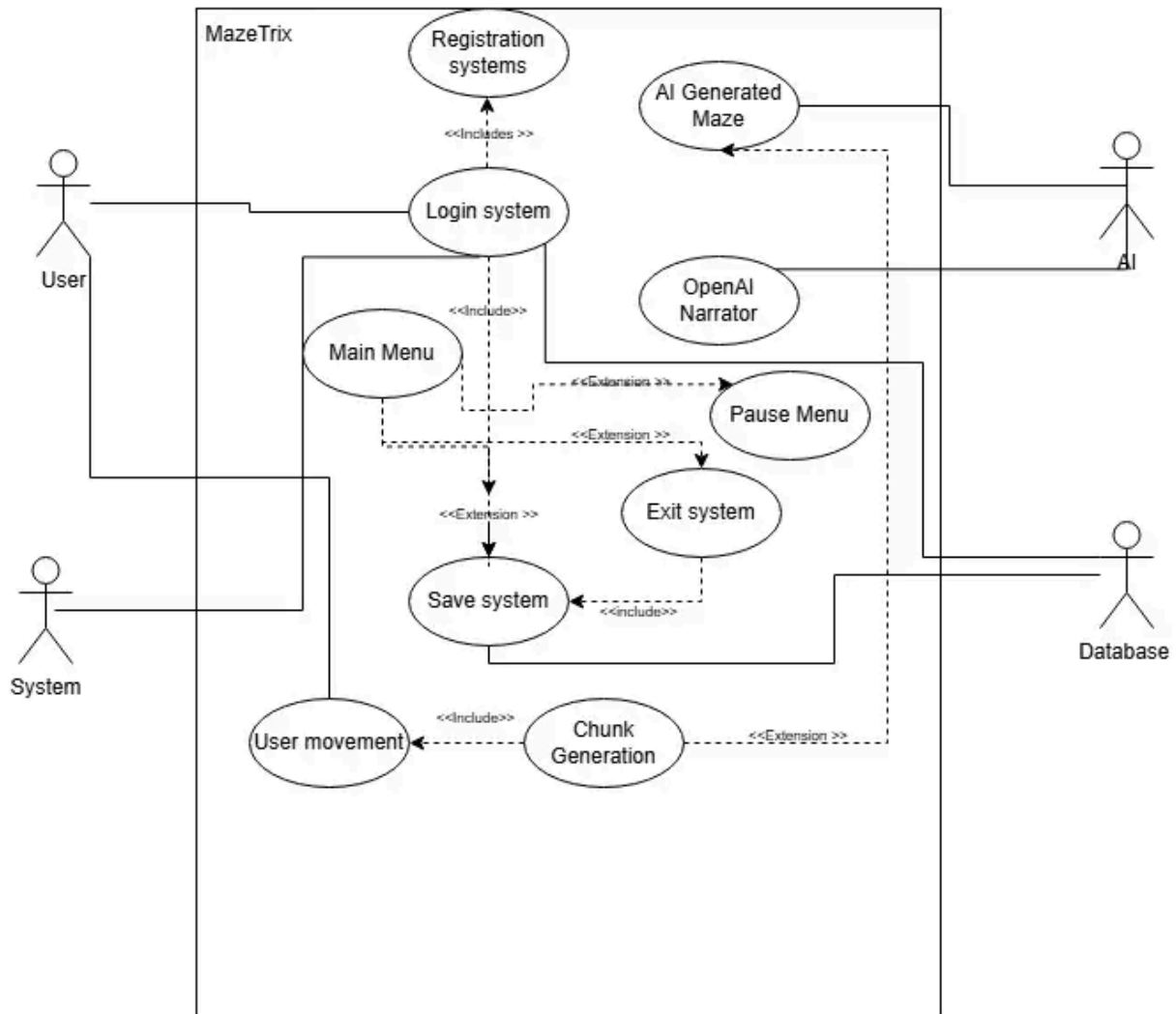
- The player collides with an item tile.
- System verifies item eligibility (e.g., health not full for potion).
- That item is removed from the map.

- Player statistics are updated accordingly:
 - + Health Potion → Restore HP
 - + Food → Restore hunger
 - + Bomb → Increase bomb inventory
 - + Speed Boost → Double movement speed for 20 seconds

Output:

- Player stats updated.
- Item removed from the environment.

5.3 Use Case Diagrams



6.0 DATABASE MANAGEMENT

For MazeTrix, we will be utilizing Google Firebase Cloud Firestore as the database management system (Firestore is a NoSQL document database). Even though Firestore isn't "tables & foreign keys" like SQL, you can still describe primary keys and relationships clearly through the uses of IDs and references.

1. users (collection)
 - Stores player/account info
 - Primary Key
 - `userId` (string) → Firestore Document ID (Firebase Auth UID)
 - Attributes
 - `email` (string)
 - `displayName` (string)
 - `createdAt` (timestamp)
 - `lastLoginAt` (timestamp)
 - `role` (string)
 - Relationships
 - 1 user has many maze runs (`maze_runs.userId` references `users.userId`)
 - 1 user has one stats document (`user_stats.userId` references `user.userId`)
2. maze_runs (collection)
 - Primary Key
 - `runId` (string) → Firestore Document ID
 - Foreign Key (fields)
 - `userId` (string) → references `users.userId`
 - `mazeId` (string) → references `mazes.mazeId`
 - Attributes
 - `status` (string) – in progress, completed, failed
 - `startTime` (timestamp)
 - `endTime` (timestamp, nullable)
 - `timeSeconds` (number)
 - `moves` (number)
 - `deaths` (number)
 - `score` (number)
 - Relationships
 - Many runs belong to one user
 - Many runs belong to one maze
3. mazes (collection)
 - Stores maze levels
 - Primary Key
 - `mazeId` (string) → Firestore Document ID
 - Attributes
 - `name` (string)
 - `difficulty` (string)
 - `width` (number)
 - `height` (number)
 - `seed` (string) – optional
 - `layout` (map/object/array) – stores maze grid data

- createdAt (timestamp)
- Relationships
 - 1 maze has many runs (maze_runs.mazeId)
- 4. user_stats (collection)
 - Precomputed stats per player
 - Primary Key
 - userId (string) → you can use the same value as the Auth UID (1:1 with users)
 - Attributes
 - bestTimeSeconds (number)
 - bestScore (number)
 - totalRuns (number)
 - totalWins (number)
 - totalDeaths (number)
 - updatedAt (timestamp)
 - Relationships
 - 1 stats doc belongs to exactly 1 user (matches users.userId)
- Relationships Summary
 - ◆ users (1) → (many) maze_runs
 - ◆ mazes (1) → (many) maze_runs
 - ◆ users (1) → (1) user_stats
- Security
 - ◆ Firestore Security Rules restrict reads/writes so users can only edit their own docs
- Indexes
 - ◆ Firestore composite indexes may be used for leaderboard queries

7.0 IMPLEMENTATION

GitHub Link: <https://github.com/kvo110/MazeTriX>

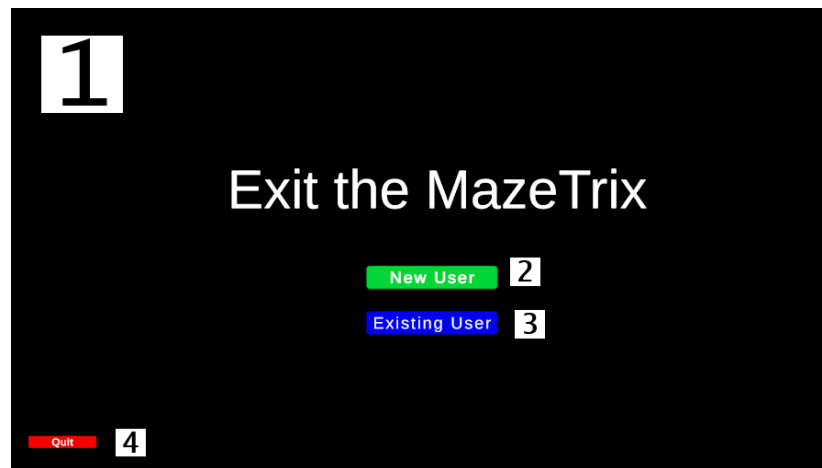
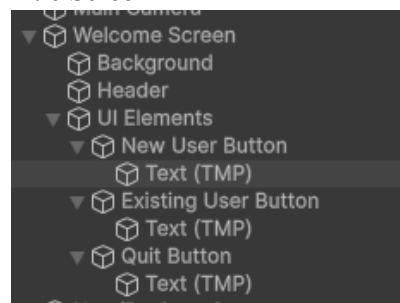
Please see the README.md for instructions on how to run the demos for each version of the interface.

7.1 Functional User Interface

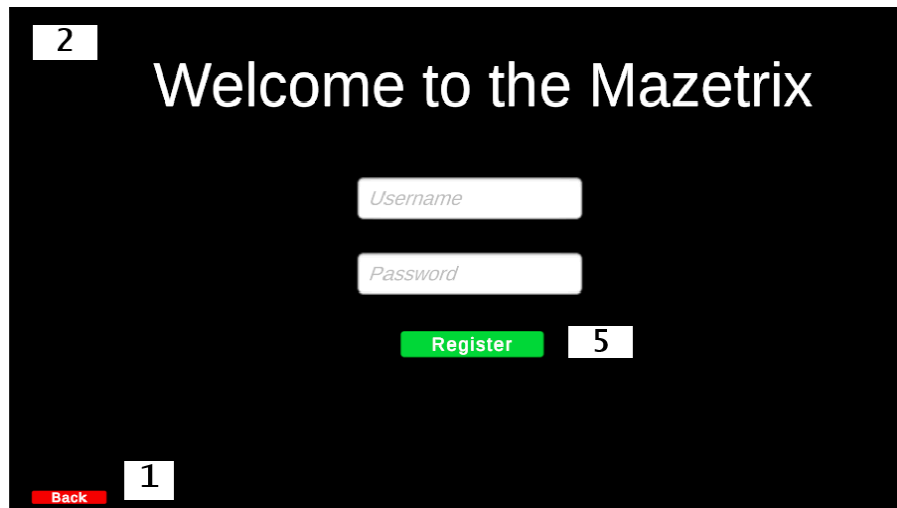
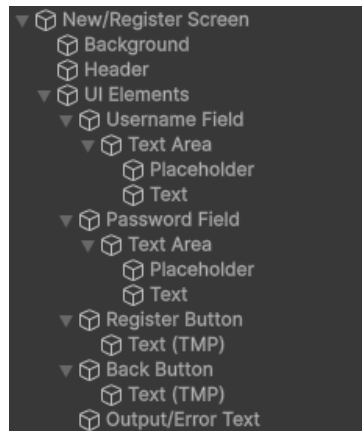
7.2 Visual Mock-Up

Created within the Unity Engine. To be integrated in the next Sprint. Numbers on the images indicate navigation to respective menus.

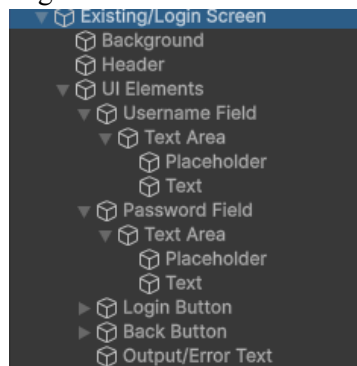
1. Title Screen



2. Registration



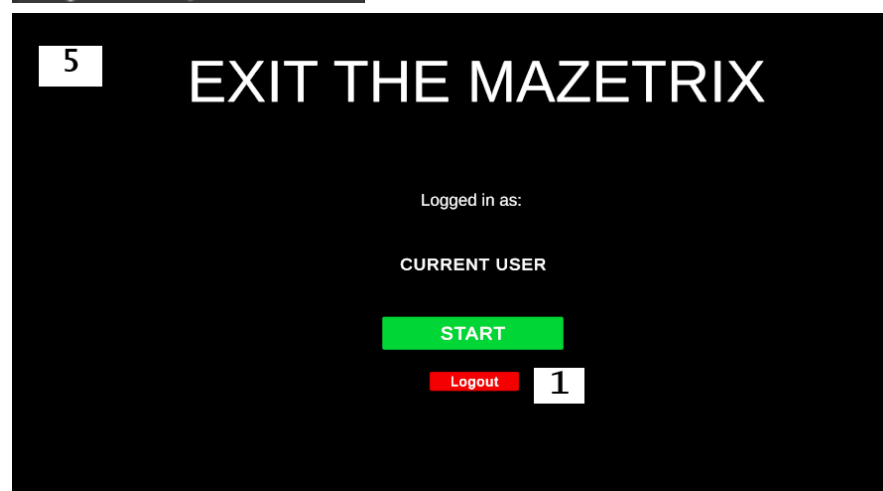
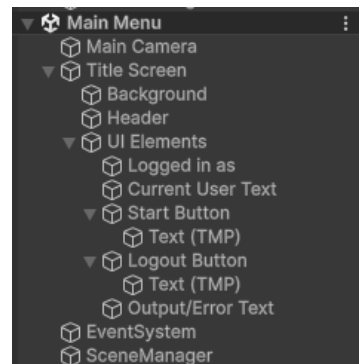
3. Login



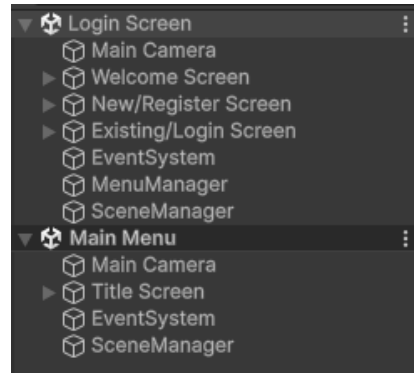


4. No associated menu. Closes the application

5. Main Menu



Full Unexpanded Hierarchy



Database saved confirmation with auto-generated User UID and Timestamp of creation

Search by email address, phone number, or user UID					Add user	↺	⋮
Identifier	Providers	Created ↓	Signed In	User UID			
kvo_test@gmail.com	✉	Feb 23, 2026	Feb 23, 2026	dCR1vVa6KuWIEpqPhRmTv5...	📄	⋮	
user@example.com	✉	Feb 16, 2026	Feb 16, 2026	TkkGADjBviOVgAOCaDT8YRX...			
Rows per page: 50 ▼					1 – 2 of 2	⏪	⏩